Sharded Blockchains

Matthieu Pigaglio

Thesis submitted in partial fulfillment of the requirements for the Degree of Doctor in Applied Sciences

October 2026

ICTEAM
Louvain School of Engineering
Université catholique de Louvain
Louvain-la-Neuve
Belgium

Thesis Committee:

Pr. Wonderful **Supervisor** (Advisor)	UCLouvain/ICTEAM, Belgium
Pr. Good **Doctor**	UCLouvain/ICTEAM, Belgium
Pr. Another **Guy**	Another University, World

Sharded Blockchains
by Matthieu Pigaglio

© Matthieu Pigaglio 2019
ICTEAM
Université catholique de Louvain
Place Sainte-Barbe, 2
1348 Louvain-la-Neuve
Belgium

This work was partially supported by the F.R.S-FNRS.

Hey, I want to
tell you a TCP joke...
ANONYMOUS

Preamble

We will put here some text that will motivate the need of your thesis. Because the world needs your input, you feel invested in a four-year long quest to fill in the gap between your dreams and the reality. In particular, the following explains why the world is wrong.

Quisque consectetuer. In suscipit mauris a dolor pellentesque consectetuer. Mauris convallis neque non erat. In lacinia. Pellentesque leo eros, sagittis quis, fermentum quis, tincidunt ut, sapien. Maecenas sem. Curabitur eros odio, interdum eu, feugiat eu, porta ac, nisl. Curabitur nunc. Etiam fermentum convallis velit. Pellentesque laoreet lacus. Quisque sed elit. Nam quis tellus. Aliquam tellus arcu, adipiscing non, tincidunt eleifend, adipiscing quis, augue. Vivamus elementum placerat enim. Suspendisse ut tortor. Integer faucibus adipiscing felis. Aenean consectetuer mattis lectus. Morbi malesuada faucibus dolor. Nam lacus. Etiam arcu libero, malesuada vitae, aliquam vitae, blandit tristique, nisl.

This thesis bridges this gap to achieve very interesting results. In particular, the main contributions of this thesis are the following.

- *A first very interesting contribution to highlight.*

  We here describe some lines to show what you first made. Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

- *A second element based on your first contribution.*

  Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas

eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

- *Now forget the first two contributions to see this one.*

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetuer. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

- *A last point to close the loop.*

Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

The thesis is organized as follows. Part I gives the background: Chapter 1 covers blockchain fundamentals, Chapter 2 the scalability problem and existing solutions, and Chapter 3 the open challenges this thesis addresses. Part II

studies the locality of Ethereum transactions, Part III presents SOAPS, and Part IV presents PANDAS. Finally, the conclusion summarizes the contributions and discusses perspectives.

Bibliographic notes

Conference Publications

1. J.-L. Picard and W. Supervisor. "A First Wonderful Paper: How Did I Made It?" In: *Proceedings of the 42nd International Conference on Making the Research Great Again – MaReGA'17*. Groville, Groland, Apr. 2017, pp. 1–6. DOI: 12.3456789/ABC.2017.123456.

2. J.-L. Picard, F. Baggins, E. Aragorn, and W. Supervisor. "Unhoped: How Such a Funny Publication Got Accepted". In: *Proceedings of the 3rd Conference of the Fellowship of the Ring – FetRi'18*. Edoras, Rohan: Elven Press, 2018, pp. 161–175. ISBN: 123-4-5678-9012-3. DOI: 23.4567/8901234.5678901.

Posters and Demos

1. Legolas and J.-L. Picard. "Bowie: A Fully-Fledged Bow". In: *Proceedings of the 21st Annual International Conference on Fantastic Weapons and Demos - FaWeDe'18*. Mirkwood, Rhovanion: Elven Press, 2018, pp. 230–232. ISBN: 712-9-4787-0000-9. DOI: 80.0000/9988875.0213456.

Journal Publications

1. J.-L. Picard, J. T. Kirk, and B. Sisko. "Voyager: Killing a Wonderful Science-Fiction Story". In: *DC Communications in Computer Sciences – DC ComiCS* 13 (2017), pp. 42–78. ISSN: 9129-8239. DOI: 00.4884/w.ploins.2019.80.012.

2. B. Wayne, J.-L. Picard, and C. Kent. "Improving the Obfuscation of Research Papers with Nonsense". In: *DC Communications in Computer Sciences – DC ComiCS* 69 (2019), pp. 114–122. ISSN: 9999-8879. DOI: 00.4444/w.fulonn.2019.00.012.

Miscellaneous Contributions

1. J.-L. Picard. *On the Impact of Turboprops on the Sea Otter Reproduction*. Invited talk at the National Turefu – NaTure'19. Pandoran, Nov. 2019.

Acknowledgments

While this version was first brought by Quentin De Coninck, it relies on the templates of numerous previous Ph.D. students, including David Lebrun, Olivier Tilmans and Fabien Duchêne.

Contents

Preamble i

Acknowledgments v

Table of Contents vii

I Background and State of the Art 1

1 Blockchain Fundamentals 3
- 1.1 Distributed Ledgers . . . 3
- 1.2 Consensus . . . 6
  - 1.2.1 Proof-of-Work . . . 6
  - 1.2.2 Proof-of-Stake . . . 7
  - 1.2.3 BFT Variants . . . 8
- 1.3 Ethereum Proof-of-Stake . . . 8
  - 1.3.1 Validators . . . 9
  - 1.3.2 Slots and Epochs . . . 9
  - 1.3.3 Committees and Attestations . . . 10
  - 1.3.4 Fork-Choice Rule . . . 10
  - 1.3.5 Rewards and Safeguards . . . 11
- 1.4 Transaction and State Models . . . 12
  - 1.4.1 UTXO Model . . . 12
  - 1.4.2 Account-Based Model . . . 13
- 1.5 Smart Contracts and Execution Environments . . . 14
  - 1.5.1 The Ethereum Virtual Machine . . . 14
  - 1.5.2 Alternative Execution Environments . . . 15

2 The Scalability Problem 17
- 2.1 Throughput, Latency, and Full-Node Cost Bottlenecks . . . 17
- 2.2 Exploiting Workload Properties . . . 18
  - 2.2.1 Parallel Transaction Execution . . . 18
  - 2.2.2 Light Nodes and Stateless Clients . . . 19
- 2.3 On-Chain Scaling: Sharding . . . 19
  - 2.3.1 Network Sharding . . . 20

2.3.2 Transaction Sharding 20
2.3.3 State Sharding 21
2.3.4 Existing Sharded Blockchains and Their Limitations . 22
2.4 Off-Chain Scaling: Layer 2 23
2.4.1 Rollups 23
2.4.1.1 Optimistic Rollups 24
2.4.1.2 Zero-Knowledge Rollups 24
2.4.2 State Channels 24
2.4.3 Sidechains 25
2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations 25

3 Synthesis and Motivation **27**
3.1 Open Challenges 27
3.2 Common Requirements for L1 and L2 Sharding 27
3.3 Positioning of the Contributions 28

Part I

Background and State of the Art

Blockchain Fundamentals 1

- Only cover what the reader needs for the rest of the thesis. No general state of the art here.
  - Check that each concept is used later on: consensus and slashing in Part III, the state model and execution in Part II, slot timing and block production in Part IV.
  - Go quickly through the generic parts and spend more pages on Ethereum, since all three contributions target it.
- Use Ethereum as the running example everywhere (workload in Part II, sharding in Part III, DAS in Part IV).
- Order of the chapter: ledgers, consensus, Ethereum PoS in detail, state models, execution.

1.1 Distributed Ledgers

A distributed ledger is an append-only log of transactions, which is replicated by a set of nodes that do not trust each other. Bitcoin [Nak08] was, in 2009, the first ledger of this kind to work in an open setting, and its transactions are transfers of a native currency. In 2015, Ethereum [Woo14] extended the model so that transactions can call programs (smart contracts) whose code and storage are kept in the replicated state. We describe here the elements of this model that are used in the rest of the thesis, and leave the question of how nodes agree on the content of the log to section 1.2.

State machine replication. A ledger can be seen as a case of state machine replication [Sch90]. Each node holds a copy of a state S (in the simplest case, the balance of every account), and a transaction tx changes it through a transition function, $S' = \delta(S, tx)$. If all nodes start from the same initial state S_0, the *genesis* state, and apply the same sequence of transactions $tx_1, \ldots, tx_n$, they all obtain

$$S_n = \delta(\ldots \delta(\delta(S_0, tx_1), tx_2) \ldots, tx_n).$$

For this to hold, δ has to be deterministic. The execution environments of blockchains that support smart contracts (section 1.5) are designed with this

in mind, and give programs no access to local clocks, floating-point arithmetic or external services, all of which could give different results on different machines. Nodes therefore never need to exchange the state to stay consistent: the state can always be recomputed from the log, and what they have to agree on is the order of transactions, which is the job of consensus.

Blocks and hash chaining. Transactions are added to the log in batches, called blocks. A block is made of a body, which holds the ordered list of transactions, and a header with metadata. The header typically contains the hash of the parent header, a timestamp, and short commitments to the content of the block, such as the root of a Merkle tree over its transactions and, in some ledgers, a commitment to the state obtained after executing the block.

Because each header includes the hash of its parent, the blocks form a hash chain back to genesis. Linking documents through hashes to detect tampering is an older idea, used for digital timestamping [HS91]. Under the usual assumption that the hash function is collision resistant, the hash of the most recent header fixes the whole history. Changing a transaction in an old block changes the hash of that block, then the parent hash stored in its child, and so on up to the latest header, so a node that knows a recent header will notice the change. Note that this only makes a rewrite detectable. Nothing stops an attacker from producing a different chain that is internally consistent, and choosing between such chains is again left to consensus (section 1.2).

Merkle trees. These commitments are usually roots of Merkle trees [Mer87]. The leaves of a Merkle tree are hashes of data items, and every inner node is the hash of its children, so the root depends on every leaf. A proof that some item is in the tree consists of the sibling hashes on the path from its leaf to the root, which is logarithmic in the number of leaves, and it can be checked by anyone who knows the root. The same technique applies to the state. If the header contains the root of a tree built over all state entries, such as account balances or the storage of contracts, then any single entry can be proven against that root. How the state is organised in practice depends on the transaction and state model of the ledger, which we discuss in section 1.4. Light clients (subsection 2.2.2) rely on such proofs to check single values against a header without holding the block or the state.

Permissioned and permissionless ledgers. Ledgers also differ in who is allowed to maintain them. In a permissioned ledger such as Hyperledger Fabric [And+18], participants are known in advance and authenticated, and classical Byzantine fault-tolerant protocols (subsection 1.2.3) can be used as they are, since these assume a fixed set of n nodes with at most f faulty ones. Public blockchains such as Bitcoin are permissionless: anyone can join or leave,

and an identity is a public key that costs nothing to generate. A single participant can then create as many identities as it wants (a Sybil attack [Dou02]), and votes cannot simply be counted per identity. Voting power is instead tied to a resource that is hard to obtain, computation in Proof-of-Work (subsection 1.2.1) or locked stake in Proof-of-Stake (subsection 1.2.2).

This thesis only considers permissionless ledgers. For our contributions, this means that the mechanisms we add, namely the placement of state on shards in Part III and the assignment of data to nodes in Part IV, have to work with open membership and with some malicious participants. We also want these decisions to be checkable by any node, and hard to bias for a participant who would benefit from a particular outcome.

Node roles. Not all nodes do the same work [CBC22]. A full node checks every new block by re-executing its transactions and verifying that the result matches the commitments in the header. It keeps the current state, and possibly a limited window of recent states. A new full node does not usually replay the chain from genesis: it starts from a recent checkpoint, downloads the corresponding state, and validates blocks from that point on. Under Proof-of-Stake, the node has to obtain this checkpoint from a source it trusts, which is related to the weak subjectivity issue discussed in subsection 1.2.2. An archive node is a full node that also keeps all historical states, so that it can answer queries such as the balance of an account at a given past block, at the price of a much larger disk footprint. We used an archive node to build the dataset of Part II, since obtaining the state accessed by an old transaction requires re-executing it on the state of that time. Validators (miners under Proof-of-Work) are full nodes that also propose and vote on blocks; under Proof-of-Stake, they are the participants that have locked stake (subsection 1.2.2). Finally, light clients only follow block headers and check individual values with Merkle proofs [CBC22]. They do not re-execute transactions, and instead assume that most consensus participants are honest (subsection 2.2.2).

Peer-to-peer network. Nodes are connected by a peer-to-peer overlay in which each node maintains a bounded number of connections. Transactions and blocks are propagated by gossip, where a node that receives a valid message forwards it to some of its neighbours. Peers are found through a discovery protocol, often a distributed hash table such as Kademlia [MM02]. To keep the bandwidth cost bounded, gossip protocols such as GossipSub [Vyz+20] send full messages only to a small set of neighbours and announce them to the others with short metadata.

The network matters for the protocol because the time it takes for a block to reach the other nodes limits how fast blocks can be produced without hurting security [GKL15; Ger+16]. Block intervals are therefore chosen to be large

compared to this delay. Part IV works at this level, since its goal is to distribute and sample data over the peer-to-peer network within the time that consensus allows.

Full replication. In the model described so far, every full node stores the whole state and re-executes every transaction of every new block. This is why a node does not need to trust others, but it also means that the capacity of the system does not grow with the number of nodes. Throughput is limited by what a single node, run by a regular participant, can download, store and execute. Larger or more frequent blocks would raise this limit, but they would also make nodes more expensive to run, so fewer participants would run them, and they would slow down block propagation, which leads to more forks and weaker security [Ger+16]. chapter 2 discusses this problem and the main approaches to it. The one this thesis builds on is sharding, where each node only handles part of the transactions and of the data.

1.2 Consensus

- What consensus has to do: get everyone to agree on one order of blocks, even with malicious nodes and an unreliable network [LSP82; FLP85].
  - Define Byzantine faults (arbitrary behaviour, collusion, equivocation).
  - Briefly recall the network models (synchronous, partially synchronous, asynchronous) and FLP [FLP85].
- Safety and liveness: no two conflicting finalized blocks, and transactions eventually get in.
  - There is a tension between the two under network partitions. Ethereum tries to get both by combining an always-available chain with a finalized prefix [NTT21].
- Sybil resistance: weight votes by something costly, either computation or stake.
- Possibly a small figure grouping the families I use later (PoW, PoS, committee BFT).

1.2.1 Proof-of-Work

- Keep this one short, mostly as history and to contrast with PoS.

- Leader election by solving a hash puzzle, and the longest/heaviest chain rule (Bitcoin, Ethereum before 2022) [SZ15].
  - Your chance of producing the next block is proportional to your share of hash power.
  - Mention GHOST (heaviest subtree rather than longest chain), because Ethereum's LMD-GHOST comes from it [SZ15].
- Finality is only probabilistic, energy cost is high, and the block interval has to stay large compared to propagation delay [GKL15].
  - The backbone analysis shows the security properties only hold if blocks propagate fast enough [GKL15].
  - Bigger or more frequent blocks produce more forks and weaken security [Ger+16]. This is a first sign of why you cannot just "turn up" throughput.

1.2.2 Proof-of-Stake

- Voting power comes from locked stake, and validators are paid for behaving correctly [Kia+17; Gil+17].
  - Two examples with security proofs: Ouroboros (longest-chain style) [Kia+17] and Algorand (committees picked by cryptographic sortition, then BFT) [Gil+17].
- Misbehaviour can be proven and punished by burning stake (slashing), so security becomes economic instead of computational [NTT21].
  - Equivocating means signing two conflicting messages, and that signed evidence can be posted on chain.
  - An attacker now risks losing their stake rather than paying for electricity.
- Proposers and committees are drawn using randomness produced on chain [Gil+17; Nag+25].
  - That randomness must be hard to predict and to bias, otherwise an attacker can pick their own slots or committees [Nag+25].
  - I need the same property in SOAPS (electing who computes the placement) and in PANDAS (assigning cells to nodes).
- One sentence each on the usual PoS problems: nothing-at-stake, long-range attacks, weak subjectivity.

1.2.3 BFT Variants

- Recall classical BFT: PBFT, $n \geq 3f + 1$, quorum certificates, and finality as soon as a decision is made [LSP82; CL99].
  - Why it works: two quorums of $2f + 1$ always share at least one honest node.
  - A leader drives the protocol in phases (pre-prepare, prepare, commit), and a view change replaces a faulty leader [CL99].
- The number of messages limits how big a committee can be: quadratic in PBFT, linear in newer protocols like HotStuff [Yin+19].
  - Signature aggregation (BLS, threshold signatures) helps a lot. Ethereum attestations use it.
- Why this matters here: sharded blockchains such as OmniLedger [Kok+18] or RapidChain [ZMR18] run BFT inside each shard committee, so the committee size vs. security question comes back in section 2.3.
  - Small committees reach agreement faster, but they are more likely to end up with too many malicious members.

1.3 Ethereum Proof-of-Stake

- Since the Merge in 2022, Ethereum runs PoS through the beacon chain, with separate consensus and execution clients [Sch+22].
  - The consensus layer (beacon chain) handles validators, committees, fork choice and finality.
  - The execution layer runs the EVM, keeps the mempool and the state, and talks to the consensus client through the Engine API.
  - Point out that the beacon chain is also what sharding designs used as their coordination chain (see section 2.3 and Part III).
- This section needs more detail than in the papers, because SOAPS (Part III) plugs into this machinery and PANDAS (Part IV) has to fit in its timing.
- Add a figure showing one slot and one epoch: proposal, attestations, aggregation, checkpoints.
- For protocol parameters, cite the consensus specs and say which upgrade introduced them (Altair, Bellatrix, Deneb, Electra, etc.).

1.3.1 Validators

- A validator deposits ETH (32 ETH originally, more allowed since Pectra) and runs both a consensus and an execution client.
  - The effective balance sets both the voting weight and the chance of being selected.
  - Mention briefly that staking pools and liquid staking put a big share of the stake in a few hands.
- What validators do: propose blocks, attest, and sometimes sit on the sync committee.
  - The proposer builds the block for its slot, or gets it from a builder.
  - Every validator attests once per epoch, voting on the head and on checkpoints.
  - Some committee members are aggregators and combine the signatures.
  - The sync committee is a small rotating group that signs headers for light clients.
- Entry and exit go through queues. Give an idea of the scale: hundreds of thousands of validators, close to a million.
  - The churn limit caps how fast the validator set changes, and so how fast an attacker can build up weight.
  - With that many validators, all-to-all BFT is out of the question, which is why committees and aggregation exist.

1.3.2 Slots and Epochs

- Time is cut into 12-second slots, and 32 slots make an epoch (about 6.4 min).
  - A slot can stay empty if the proposer misses it. Time keeps going anyway.
- Each slot has one proposer, picked at random using RANDAO, which accumulates randomness on chain over the previous epochs [Nag+25].
  - Every proposer mixes in a signature. The last proposers of an epoch can bias the result by not publishing their block [Nag+25].
  - Proposers are known ahead of time. Useful for networking, but it also makes them easy targets.

- Walk through what happens inside a slot, because the 4 s mark is the whole constraint of PANDAS (Part IV).
  - At 0 s the block is proposed and propagated, at 4 s attesters vote for the head they see, at 8 s aggregates go out.
  - So everything an attester checks before voting (validity, and with DAS also availability) has to be done within those first 4 s.
  - For what confirmation times look like in practice, see Pacheco et al. [Pac+23].

1.3.3 Committees and Attestations

- Each epoch, validators are shuffled into committees, one set per slot, using the RANDAO seed. Each validator attests once per epoch.
  - The shuffle is deterministic and public: anyone can recompute it and check that a validator really belongs to a committee.
  - Committees have a minimum size so that a dishonest majority in one of them is very unlikely.
- An attestation votes for the chain head (LMD-GHOST) and for checkpoints (Casper FFG) [Sch+22], and signatures are aggregated with BLS.
  - Concretely it carries a head vote, a source checkpoint and a target checkpoint.
  - Without BLS aggregation the bandwidth would not hold.
- I want to stress this mechanism because both my designs reuse the idea: shard committees in Part III, and which node stores which cells in Part IV.
  - The idea is that assignments are computed from public randomness, so they are deterministic and anyone can verify them. This comes back throughout the thesis.

1.3.4 Fork-Choice Rule

- Gasper combines LMD-GHOST, which picks the head and keeps the chain live, with Casper FFG, which gives finality and safety [Sch+22; NTT21].
  - LMD-GHOST: start at the last justified checkpoint and, at every fork, follow the branch backed by the most recent attestations.
  - Casper FFG sits on top and works on epoch checkpoints.

  - Neu et al.'s ebb-and-flow framing is a nice way to explain it: a chain that is always available, plus a finalized prefix [NTT21].

- A checkpoint is justified with 2/3 of the stake and finalized one epoch later, so roughly two epochs in total.
  - Reverting a finalized block requires at least 1/3 of the stake to get slashed.
  - Mention the known attacks on the fork choice (balancing, ex-ante reorgs) and the fix that was adopted (proposer boost) [Sch+22].
- Validators should only vote for blocks that are valid and whose data is *available*. With DAS they check availability by sampling before voting (tight fork choice) [Pig+25]. Part IV builds on this.
  - With a tight fork choice, a block whose data cannot be retrieved never becomes the head, even if it is valid.
  - The alternative is to sample later (trailing fork choice), which gives weaker guarantees. PANDAS targets the tight version.

1.3.5 Rewards and Safeguards

- Validators earn rewards for timely and correct attestations and proposals, and lose a bit when they miss duties.
  - The reward depends on how fast and how correct the head, source and target votes are.
  - Proposers also earn something for including attestations and slashing evidence.
- Slashing covers provable equivocation: two proposals for the same slot, two votes for the same target, or surround votes [Sch+22].
  - The slashed validator pays an initial penalty, is forced out with a long withdrawal delay, and then pays a correlation penalty that grows with how much stake was slashed around the same time. One mistake is cheap, but a coordinated attack is very expensive.
  - Whoever reports the evidence gets a small reward.
- The inactivity leak drains offline validators when more than 1/3 of the stake stops participating, so finality can resume [PAT24].
  - Pavloff et al. show that Byzantine validators can turn these penalties against honest ones [PAT24], so incentives need care.

- Main point for Part III: Ethereum enforces honest behaviour through money. SOAPS uses the same tools (election, deposits, slashing) instead of adding its own consensus.
  - Only behaviour that can be proven can be slashed. So any deviation in SOAPS has to be detectable and attributable to someone, which is why it has a challenge mechanism.
- Finish with proposer-builder separation (MEV-Boost, builders) [Dai+20; Hei+23], since Part IV needs to know who seeds the blob data.
  - Explain MEV first: the profit that comes from reordering, inserting or censoring transactions [Dai+20].
  - In practice builders bid in auctions and relays pass their blocks to proposers. A handful of builders produce most blocks [Hei+23].
  - For Part IV this means the initial data comes from a few well-provisioned builders, while many other nodes are modest machines.

1.4 Transaction and State Models

- The state model decides what "the data a transaction accesses" actually is. That in turn decides how state can be split across shards and what counts as a conflict (parallel execution, locality in Part II).
- There are two main models: UTXO (Bitcoin and similar) and accounts (Ethereum and similar). This thesis works with accounts.

1.4.1 UTXO Model

- The state is the set of unspent outputs. A transaction consumes some outputs and creates new ones (Bitcoin).
  - Each output can be spent once, so a double spend is two transactions consuming the same output.
  - A small figure would help: one transaction, two inputs, two outputs.
- Inputs are listed explicitly, so dependencies are known before execution. This makes partitioning and parallel validation easy.
  - Two transactions conflict only if they share an input, so independent ones can be checked in parallel without any trouble.

- The downside is that general smart contracts are awkward to write. There is work on sharding and placement for UTXO [Ngu+19], but state is split differently, so I leave it out of scope.
  - Extended UTXO models add scripts and data to outputs while keeping the explicit inputs.

1.4.2 Account-Based Model

- The state is a key/value store of accounts: user accounts (balance, nonce) and contract accounts (code, storage).
  - User accounts (EOAs) are controlled by a private key. The nonce orders their transactions and prevents replays.
  - Contract accounts hold code and their own storage, a map of 256-bit words.
- A transaction is either a transfer or a contract call. A call can trigger more calls and end up touching many accounts and storage slots [PKR23; GDW19].
  - These internal calls do not appear in the block. You only see them in execution traces.
  - The Ethereum transaction graph is very skewed: a few accounts (exchanges, token contracts, DeFi protocols) interact with a large share of everyone else [GDW19].
- In general you do not know what state a transaction will touch until you run it. This motivates the measurements in Part II and makes placement hard in Part III.
  - The same call can read different storage depending on the current state.
  - So whether two transactions conflict is only known after running them.
- Gas: every operation has a cost, and fees are proportional to gas used (EIP-1559 base fee plus tip) [Rou24; Pac+23].
  - The block gas limit caps how much work fits in one block.
  - With EIP-1559 the protocol sets a base fee that follows demand and is burned, and users add a tip for the proposer [Rou24].
  - The gas limit is basically the throughput knob, and it is held back by what small nodes can process in time.

1.5 Smart Contracts and Execution Environments

- Smart contracts are programs stored on chain and run in the same way by every full node.
  - Give the usual examples: tokens (ERC-20, ERC-721), DEXs, lending, bridges. They make up most of the workload in Part II.
- Contracts call other contracts, and that is where the dependencies between accounts studied in Part II come from.
  - A single swap can go through several pools and touch tens of contracts.
  - Popular contracts end up as hotspots that many transactions touch.

1.5.1 The Ethereum Virtual Machine

- The EVM is a deterministic stack machine, and transactions in a block run one after the other.
  - Describe it quickly: 256-bit words, temporary memory, persistent storage. Storage reads and writes (SLOAD/SSTORE) are by far the most expensive.
  - Contracts call each other with CALL, DELEGATECALL and STATICCALL. A revert undoes what the call changed.
- How state is stored: one Merkle Patricia Trie for accounts, one storage trie per contract, and the root goes in the block header.
  - Reading a value means walking down the trie. Disk access is one of the main costs of execution, so caching recently used state pays off (temporal locality).
- Explain how you get internal calls and storage accesses out of execution traces, since Part II depends on it [PKR23].
  - The method is to replay old blocks on an archive node with tracers, to get the exact read and write sets.
  - Doing that for the whole history is expensive, which is why Part II spends time on the engineering.
- Sequential execution on a single global state is what keeps Ethereum L1 at roughly 15 to 30 transactions per second.

1.5.2 Alternative Execution Environments

- Many chains and most L2s are EVM-compatible: same semantics, deployed somewhere else.
  - They keep the EVM so contracts and tooling carry over, and as a result their workloads look a lot like Ethereum's.
- Some environments were designed for parallel execution from the start: Solana (Sealevel), Sui [Bla+24], Aptos with Block-STM [Gel+23].
  - On Solana each transaction lists the accounts it will read and write, and the runtime runs non-overlapping ones in parallel.
  - On Sui, transactions on owned objects skip consensus and are just broadcast. Only shared objects need ordering [Bla+24].
  - Aptos runs the block speculatively in parallel and re-runs transactions when it finds a conflict, keeping the block order [Gel+23].
- Either you declare dependencies up front or you bet there are few of them. Both only pay off if the workload cooperates (section 2.2).

The Scalability Problem | 2

- Here I explain why blockchains do not scale, present the two families of solutions (sharding on L1, and L2s), and for each one say clearly what is still missing.
- Much of this can come from the related work sections of the locality, SOAPS and PANDAS papers, extended.
- Order: bottlenecks, optimizations that rely on the workload, sharding, L2 and data availability, and the limitations of each.
- At the end, a table comparing existing approaches on a few criteria: what gets sharded, whether it uses the workload, whether assignments can be verified, whether it is part of consensus, and whether it was evaluated on a real workload.

2.1 Throughput, Latency, and Full-Node Cost Bottlenecks

- Throughput is limited because every node re-runs every transaction. Compare with centralized payment systems to give an order of magnitude [Ger+16].
  - Bitcoin and Ethereum L1 handle tens of transactions per second, card networks thousands.
  - Bigger blocks or a higher gas limit mean more expensive nodes and slower propagation [Ger+16].
- Latency depends on the block interval and on how long finality takes, both limited by propagation and consensus rounds.
  - On Ethereum a transaction gets in within a 12 s slot, and is final after about two epochs, so around 13 minutes [Pac+23].
- Running a full node costs storage (the state keeps growing), bandwidth (blocks and data) and CPU. If these go up, small participants drop out and the network becomes less decentralized [Ger+16].

  - The state already holds hundreds of millions of accounts, and the history is much bigger.
  - For publishing data, bandwidth is what runs out first. This is the motivation for Part IV.

- I can mention the trilemma here, but only briefly, as context. It is not what drives the thesis.
- The message I want to get across: L1 and L2 both need to scale, and the general answer is sharding, meaning that no single node handles every transaction or every piece of data.
  - Sharding computation means not every node executes every transaction (Part III).
  - Sharding data means not every node downloads every piece of data (Part IV).

2.2 Exploiting Workload Properties

- Some optimizations leave the protocol alone and instead count on the workload behaving well: few conflicts, and state that gets reused.
- I introduce them here because Part II checks whether those assumptions actually hold on Ethereum.
- Introduce the two kinds of locality informally (Part II gives the formal definitions):
  - spatial locality, or disjoint access parallelism: transactions running at the same time touch different state;
  - temporal locality: state used recently is likely to be used again soon.

2.2.1 Parallel Transaction Execution

- The idea is to run transactions that do not conflict at the same time inside a block.
  - The result must still be the same as running them one by one in block order.
- Three ways to do it: declare access lists up front (EIP-2930, Solana), run optimistically and redo on conflict (Block-STM [Gel+23]), or have the proposer compute a schedule (Dickerson et al. [Dic+17]).

  - With declared access lists, declaring too much loses parallelism and declaring too little makes the transaction fail.
  - The optimistic approach needs nothing from users, but it slows down when there are many conflicts [Gel+23].
  - In Dickerson et al., the proposer runs the block speculatively and publishes a schedule that validators can replay in parallel [Dic+17].

- How well any of this works depends on how many transactions conflict, which is the spatial locality measured in Part II.

- I do not contribute to parallel execution myself. I mention it because Part II's results say something about it, and I come back to it only as a perspective.

2.2.2 Light Nodes and Stateless Clients

- Light clients check headers and Merkle proofs instead of keeping the whole state [CBC22].
  - They trust that the majority of the consensus (in Ethereum, of the sync committee) is honest. On their own they cannot spot an invalid block or missing data [CBC22].
  - Fraud proofs and data availability sampling were proposed to fix that [Al-+21]. This leads into subsection 2.4.4.

- Stateless clients go further: blocks come with witnesses for the state they touch, and caching recently used state makes those witnesses smaller.
  - Witness size costs bandwidth, and the trie design (Merkle Patricia vs. Verkle or binary trees) changes how big the proofs are.

- How well this works depends on how often state is reused, which is the temporal locality measured in Part II.

2.3 On-Chain Scaling: Sharding

- The principle: split nodes, state and transaction processing over several chains running in parallel (shards), each with its own committee, and coordinate them through a beacon chain [Wan+19].
  - In the ideal case, throughput grows linearly with the number of shards.

  - The beacon chain keeps the validator list and the randomness, assigns validators to shards, and records checkpoints (cross-links) of shard blocks.

- Security: a small committee is easier to take over, which is why nodes are assigned at random and reshuffled regularly [Ava+23].
  - Show how the chance of a corrupted committee depends on committee size and on the overall share of malicious nodes (hypergeometric tail). Committees are made big enough for this to be negligible.
  - Avarikioti et al. give a formal definition of what robust sharding should guarantee, including against adaptive adversaries [Ava+23].
- Following Wang et al. [Wan+19], split the discussion into network, transaction and state sharding.
- Reuse the SOAPS figure that shows a sharded blockchain with an intra-shard and a cross-shard transaction.

2.3.1 Network Sharding

- This is about splitting nodes into committees (node placement).
  - A node then only takes part in consensus and gossip for its own shard.
- Assignment uses randomness that can be verified: RandHound in OmniLedger [Kok+18], the Cuckoo rule in RapidChain [ZMR18], RANDAO in Ethereum [Nag+25]. The goal is to resist Sybil and adaptive attackers.
  - Elastico creates identities with PoW puzzles [Luu+16], while PoS designs use stake.
  - Reshuffling every epoch leaves an adaptive attacker little time to corrupt a committee, but every move forces nodes to download the state of their new shard.

2.3.2 Transaction Sharding

- Transactions are usually sent to the shard of the sender or caller [Kok+18; ZMR18].
  - It is simple, everyone knows the rule, and clients need no extra information.

- Each shard has its own mempool. Other routing rules have been proposed but are uncommon.
  - Where a transaction goes only matters together with where its state lives: it is cheap only if all its state is in the same shard.

2.3.3 State Sharding

- Each shard keeps only part of the state. Where each account lives must be known by everyone and the load must be balanced.
  - Everyone needs to know where an account lives in order to route transactions to it.
  - No shard should become the bottleneck, either for storage or for processing.
- Most designs hash the account address modulo the number of shards: Chainspace [Al-+18], Monoxide [WW19], Elastico [Luu+16], and the Ethereum sharding proposals.
  - No metadata to store and a uniform spread, but it completely ignores which accounts talk to each other.
- Intra-shard vs. cross-shard transactions. Cross-shard ones need an atomic commit across shards, typically two-phase commit (lock, vote, commit or abort), which costs extra consensus rounds and messages [Kok+18].
  - OmniLedger (Atomix): the client locks inputs in each shard, then unlocks or commits [Kok+18].
  - RapidChain splits a cross-shard transaction into sub-transactions that go through the output shard [ZMR18].
  - Monoxide uses relay transactions and only gets eventual atomicity [WW19]. BrokerChain uses broker accounts whose state is spread over several shards [Hua+22].
  - Whatever the protocol, you pay extra latency and messages, state stays locked for a while, and some transactions abort.
- With hash placement, most Ethereum transactions become cross-shard. Part III gives the actual number.
  - Quick intuition: with k shards, two random accounts land in the same shard with probability $1/k$, and a transaction touching more accounts is almost never local.

2.3.4 Existing Sharded Blockchains and Their Limitations

- Go through the main designs: OmniLedger [Kok+18], RapidChain [ZMR18], Chainspace [Al-+18], Monoxide [WW19], Elastico [Luu+16], with Wang et al.'s survey [Wan+19] as a starting point. Then tell how Ethereum dropped execution sharding and moved to rollups plus data sharding [Pig+25].
  - Put them in a table: node placement, state placement, transaction placement, cross-shard protocol.
  - Some sharded chains are running outside Ethereum (NEAR, MultiversX). Mention them without going into detail.
- First problem: hash placement does not look at the workload at all, so the cross-shard ratio is very high [Kró+21].
- A lot of work tries to place state better and cuts the number of cross-shard transactions:
  - The idea comes from databases. Schism partitions tuples with METIS on the graph of what is accessed together [Cur+10; KK98], and Mehta et al. re-shard adaptively in an untrusted setting [Meh+25].
  - Graph-based placement: TxAllo [ZPY23], BrokerChain [Hua+22]. Community detection is also common.
  - Learning-based: Spring uses deep reinforcement learning [Li+24].
  - At the transaction level, OptChain picks a shard for each transaction to avoid cross-shard ones [Ngu+19].
  - They all report big drops in the cross-shard ratio on Ethereum traces.
- Second problem: they give an explicit location to *every* account. At Ethereum's scale that is hundreds of MB of metadata that every node must keep in sync.
- Third problem: a single coordinator decides the placement, or the paper does not say who does. This opens the door to centralization and to people optimizing for themselves, and none of it is tied to consensus.
  - Usually nobody says who runs the optimizer, on what input, or how the others can check the result.
  - Whoever controls placement could favour their own accounts (cheaper, faster) or put victims in the same shard to attack them.

- Another line of work adjusts things at runtime, moving accounts around (Król et al. [Kró+21], Huang et al. [HLZ24], BIND [Qi+26]) or scheduling transactions together (Concordia [Liu+26]). They pay for it in migration or scheduling overhead.
  - Migration locks state while it moves [HLZ24]. BIND manages to keep processing transactions during the move [Qi+26].
  - Concordia has shards cooperate when scheduling to avoid cross-shard conflicts [Liu+26].
- What is missing is a placement that stays small, that anyone can check, and that is part of consensus. That is what Part III does.

2.4 Off-Chain Scaling: Layer 2

- The principle: run transactions off L1 but keep some or all of its security, using L1 for settlement and for publishing data [Gud+20].
  - Gudgeon et al. sort them into channels, sidechains/commit-chains and rollups [Gud+20].
  - A good way to compare them is to ask what an honest user needs to get their funds back if the operator cheats.
- L2s do not make L1 scaling unnecessary. They move the pressure onto publishing data on L1.
  - That is Ethereum's "rollup-centric" roadmap: L1 becomes the place where rollups settle and publish their data.

2.4.1 Rollups

- A sequencer orders and executes transactions off chain, then posts compressed transaction data and state roots to L1 [Gud+20].
  - A bridge contract on L1 holds the deposited funds and receives the state roots.
  - Today the sequencer is usually a single operator. Users can still force their transaction in through L1 if it censors them.
- Everything relies on the data being *available*: anyone must be able to rebuild the L2 state from it [Al-+21].
- Use one concrete example, Optimism (or Arbitrum [Kal+18]). Keep it short here and go into detail in Part IV.

  - In Optimism, the sequencer makes L2 blocks, a batcher posts them to L1 (as blobs now), and a proposer posts output roots. Anyone can rebuild the L2 chain from L1 data alone.
  - A figure would help: L2 users, sequencer, batches/blobs on L1, verifiers rebuilding the state.

2.4.1.1 Optimistic Rollups

- State roots are accepted unless someone challenges them with a fraud proof during a challenge window of about 7 days [Kal+18; Gud+20].
  - In Arbitrum the two parties bisect the disputed execution until only one instruction is left, which L1 then checks [Kal+18].
  - The price of being optimistic is the long withdrawal delay.
- Challengers need the data to recompute the disputed state.
  - If the data is withheld, nobody can write a fraud proof, and an invalid root goes through.

2.4.1.2 Zero-Knowledge Rollups

- Each batch comes with a validity proof (SNARK/STARK) checked on L1. Finality is fast, but proving is expensive [Gud+20].
  - Proving EVM execution is costly, and zkEVMs give up some compatibility to bring that cost down.
- They still need data availability, so that users can rebuild the state and withdraw.
  - A valid proof tells you the new state is correct, not what it contains. Users still need the data to know what they own.

2.4.2 State Channels

- A fixed group of participants exchange updates off chain and settle on chain at the end, as in the Lightning Network [Gud+20].
  - The channel is opened on chain, signed state updates are exchanged off chain, and it is closed on chain. In a dispute, the latest signed state wins.
  - Payment networks route a payment across several channels.
- Limitations: funds are locked, participants have to stay online, and the set of participants is fixed, so channels are not general-purpose.

2.4.3 Sidechains

- These are separate chains with their own consensus, connected to L1 by a bridge (e.g., Polygon PoS) [Gud+20].
  - Two-way peg: lock assets on L1 to mint them on the sidechain, and the other way round.
- They do not get L1's security, and bridges have been hacked many times.
  - Funds are only as safe as the sidechain validators or the bridge committee.
- Use the comparison with rollups to explain why Ethereum's roadmap went with rollups.

2.4.4 Existing Layer 2 and Data Availability Solutions and Their Limitations

- The data availability problem: how do you check that published data can be retrieved without downloading all of it [Al-+21]?
  - You cannot prove that someone withheld data, so you cannot slash for it. This is what makes it different from the other faults seen so far.
  - Light clients need availability guarantees for fraud proofs to be useful [Al-+21].
- EIP-4844 (blobs) [Pig+25] gives rollups their own cheaper, temporary space for data. But every node still downloads every blob, so capacity is capped by node bandwidth.
  - Blobs are large pieces of data attached to blocks, committed with KZG, not readable by the EVM, and deleted after a few weeks.
  - They have their own fee market and a small target and maximum number per block.
- Data availability sampling: erasure-code the data and let each node sample a few random pieces. This gives probabilistic guarantees while each node downloads only a small part (details in Part IV) [Al-+21; KZG10].
  - With erasure coding, any large enough fraction of the data is enough to rebuild all of it, so an attacker has to hide a lot to cause harm.

  - If that much is hidden, a few random samples are enough to notice with high probability.
  - Then explain the two ways to make sure the encoding is correct: fraud proofs [Al-+21], or KZG commitments, which rule out bad encoding by construction [KZG10].

- There are also separate DA layers (Celestia, EigenDA, Avail), with their own trust assumptions and timing.
  - A rollup that uses them no longer gets Ethereum's guarantees on its data.
- On the networking side, the options are GossipSub for dissemination and a DHT (Kademlia) for retrieval. PeerDAS is the first deployed step.
  - With gossip you need one topic per column or row. That does not scale well with many topics, and duplicate messages waste bandwidth.
  - DHT lookups take several hops, which is too slow for 4 s, and DHTs are exposed to Sybil and eclipse attacks.
  - PeerDAS has nodes keep a subset of columns and sample from their peers, but at a much smaller scale than the full design.
- What is missing: a way to spread and sample the data before the 4 s attestation deadline, at full Danksharding scale, while coping with faulty nodes and nodes that see different views of the network.
- This is what Part IV addresses [Pig+25].

Synthesis and Motivation | 3

- A short chapter, a few pages, to close Part I: go from the limitations of chapter 2 to the research questions.

3.1 Open Challenges

- C1: the designs above assume the workload has good locality, but this is rarely checked on real, recent data. This is Part II [PKR23].
  - Sharding, parallel execution and stateless clients are mostly evaluated on synthetic or short traces.
  - The question: can solid assumptions about the near future be drawn from historical data?
- C2: state placement has to cut cross-shard transactions while keeping its metadata small, staying verifiable, and not letting anyone tilt it in their favour. This is Part III.
  - It should fit into the existing consensus and its safeguards, without a trusted coordinator.
- C3: L2 data availability has to be checked by sampling, within consensus timing, over an open P2P network. This is Part IV.
  - The constraints: a hard 4 s deadline, tens of thousands of nodes, some of them faulty or unresponsive, and no new consensus step.

3.2 Common Requirements for L1 and L2 Sharding

- Verifiable: any node must be able to recompute who is assigned what (accounts to shards, cells to nodes), with no trusted third party.
  - This uses the same trick as Ethereum committees: deterministic functions of public randomness and public state (subsection 1.3.3).
- Flexible: able to adapt in large, changing systems (the workload evolves, nodes join and leave, some fail).

  - Fully deterministic schemes are too rigid, and fully ad-hoc ones cannot be checked. My designs try to sit in between.

- Decentralized: no heavy new consensus step, low cost per node, anyone can participate.
- Practical: evaluated on real Ethereum workloads and timing, and compatible with the existing protocols.

3.3 Positioning of the Contributions

- SOAPS shards computation on L1. It removes contention on execution and state, and it benefits from knowing the workload (Part II).
- PANDAS [Pig+25] shards the L2 data. It removes the bandwidth bottleneck for data availability, and it barely depends on the workload.
- The two complement each other rather than compete: a sharded L1 still has to host L2 data, and an L1 full of rollups still gains from parallel execution.
- A figure would help: L1 vs. L2 on one axis, computation vs. data on the other, with Parts II to IV placed on it.
- End with a short roadmap of Parts II to IV.

Bibliography

[Al-+18] M. Al-Bassam, A. Sonnino, S. Bano, D. Hrycyszyn, and G. Danezis. "Chainspace: A Sharded Smart Contracts Platform". In: *NDSS*. 2018.

[Al-+21] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi. "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2021, pp. 279–298.

[And+18] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, et al. "Hyperledger Fabric: A Distributed Operating System for Permissioned Blockchains". In: *ACM EuroSys*. 2018.

[Ava+23] Z. Avarikioti, A. Desjardins, L. Kokoris-Kogias, and R. Wattenhofer. "Divide & scale: Formalization and roadmap to robust sharding". In: *International Colloquium on Structural Information and Communication Complexity*. Springer. 2023, pp. 199–245.

[Bla+24] S. Blackshear et al. "Sui Lutris: A Blockchain Combining Broadcast and Consensus". In: *ACM CCS*. 2024, pp. 2606–2620.

[CBC22] P. Chatzigiannis, F. Baldimtsi, and K. Chalkias. "SoK: Blockchain Light Clients". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 615–641.

[CL99] M. Castro and B. Liskov. "Practical Byzantine Fault Tolerance". In: *USENIX OSDI*. 1999, pp. 173–186.

[Cur+10] C. Curino, E. P. C. Jones, Y. Zhang, and S. R. Madden. "Schism: a workload-driven approach to database replication and partitioning". In: *Proc. VLDB Endow.* (2010).

[Dai+20] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. "Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability". In: *IEEE S&P*. 2020, pp. 910–927.

[Dic+17] T. Dickerson, P. Gazzillo, M. Herlihy, and E. Koskinen. "Adding Concurrency to Smart Contracts". In: *ACM PODC*. 2017, pp. 303–312.

[Dou02] J. R. Douceur. "The Sybil Attack". In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 251–260.

[FLP85] M. J. Fischer, N. A. Lynch, and M. S. Paterson. "Impossibility of Distributed Consensus with One Faulty Process". In: *J. ACM* 32.2 (1985), pp. 374–382.

[GDW19] D. Guo, J. Dong, and K. Wang. "Graph structure and statistical properties of Ethereum transaction relationships". In: *Inf. Sci.* 492 (2019), pp. 58–71.

[Gel+23] R. Gelashvili, A. Spiegelman, Z. Xiang, G. Danezis, Z. Li, D. Malkhi, Y. Xia, and R. Zhou. "Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing". In: *ACM PPoPP*. 2023, pp. 232–244.

[Ger+16] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun. "On the Security and Performance of Proof of Work Blockchains". In: *ACM CCS*. 2016, pp. 3–16.

[Gil+17] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich. "Algorand: Scaling Byzantine Agreements for Cryptocurrencies". In: *ACM SOSP*. 2017, pp. 51–68.

[GKL15] J. Garay, A. Kiayias, and N. Leonardos. "The Bitcoin Backbone Protocol: Analysis and Applications". In: *EUROCRYPT*. 2015, pp. 281–310.

[Gud+20] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais. "SoK: Layer-Two Blockchain Protocols". In: *Financial Cryptography and Data Security (FC)*. 2020, pp. 201–226.

[Hei+23] L. Heimbach, L. Kiffer, C. Ferreira Torres, and R. Wattenhofer. "Ethereum's Proposer-Builder Separation: Promises and Realities". In: *ACM IMC*. 2023, pp. 406–420.

[HLZ24] H. Huang, Y. Lin, and Z. Zheng. "Account migration across blockchain shards using fine-tuned lock mechanism". In: *IEEE INFOCOM 2024*. 2024, pp. 271–280.

[HS91] S. Haber and W. S. Stornetta. "How to Time-Stamp a Digital Document". In: *Journal of Cryptology* 3.2 (1991), pp. 99–111.

[Hua+22] H. Huang, X. Peng, J. Zhan, S. Zhang, Y. Lin, Z. Zheng, and S. Guo. "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding". In: *IEEE INFOCOM*. 2022, pp. 1968–1977.

[Kal+18] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten. "Arbitrum: Scalable, private smart contracts". In: *USENIX Security*. 2018, pp. 1353–1370.

[Kia+17] A. Kiayias, A. Russell, B. David, and R. Oliynykov. "Ouroboros: A Provably Secure Proof-of-Stake Blockchain Protocol". In: *CRYPTO*. 2017, pp. 357–388.

[KK98] G. Karypis and V. Kumar. "A fast and high quality multilevel scheme for partitioning irregular graphs". In: *SIAM J. Sci. Comput.* 20.1 (1998), pp. 359–392.

[Kok+18] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford. "OmniLedger: A Secure, Scale-Out, Decentralized Ledger via Sharding". In: *IEEE S&P*. 2018, pp. 583–598.

[Kró+21] M. Król, O. Ascigil, S. Rene, A. Sonnino, M. Al-Bassam, and E. Rivière. "Shard scheduler: Object placement and migration in sharded account-based blockchains". In: *ACM AFT*. 2021, pp. 43–56.

[KZG10] A. Kate, G. M. Zaverucha, and I. Goldberg. "Constant-Size Commitments to Polynomials and Their Applications". In: *ASIACRYPT*. 2010, pp. 177–194.

[Li+24] P. Li, M. Song, M. Xing, Z. Xiao, Q. Ding, S. Guan, and J. Long. "Spring: Improving the throughput of sharding blockchain via deep reinforcement learning based state placement". In: *ACM Web Conf.* 2024, pp. 2836–2846.

[Liu+26] Y. Liu, L. Jia, X. Yang, Z. Li, and Y. Sun. "Concordia: Enabling Low-Conflict Distributed Transaction Scheduling in Sharding Blockchain via Cooperative Perception". In: *ACM Web Conf.* 2026, pp. 5493–5502.

[LSP82] L. Lamport, R. Shostak, and M. Pease. "The Byzantine Generals Problem". In: *ACM Trans. Program. Lang. Syst.* 4.3 (1982), pp. 382–401.

[Luu+16] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena. "A Secure Sharding Protocol For Open Blockchains". In: *ACM CCS*. 2016, pp. 17–30.

[Meh+25] B. Mehta, N. Baghel, M. J. Amiri, B. T. Loo, and R. Marcus. "Adaptive Sharding in Untrusted Environments". In: *Proc. ACM Manag. Data* 3.6 (2025), pp. 1–28.

[Mer87] R. C. Merkle. "A Digital Signature Based on a Conventional Encryption Function". In: *Advances in Cryptology (CRYPTO)*. 1987, pp. 369–378.

[MM02] P. Maymounkov and D. Mazières. "Kademlia: A Peer-to-Peer Information System Based on the XOR Metric". In: *Peer-to-Peer Systems (IPTPS)*. 2002, pp. 53–65.

[Nag+25] Á. Nagy, J. Tapolcai, I. A. Seres, and B. Ladóczki. "Forking the RANDAO: Manipulating Ethereum's Distributed Randomness Beacon". In: *ACM CCS*. 2025, pp. 873–887.

[Nak08] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. `https://bitcoin.org/bitcoin.pdf`. 2008.

[Ngu+19] L. N. Nguyen, T. D. T. Nguyen, T. N. Dinh, and M. T. Thai. "OptChain: Optimal Transactions Placement for Scalable Blockchain Sharding". In: *IEEE ICDCS*. 2019, pp. 525–535.

[NTT21] J. Neu, E. N. Tas, and D. Tse. "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma". In: *IEEE S&P*. 2021, pp. 446–465.

[Pac+23] M. Pacheco, G. Oliva, G. K. Rajbahadur, and A. Hassan. "Is my transaction done yet? an empirical study of transaction processing times in the ethereum blockchain platform". In: *ACM Trans. Softw. Eng. Methodol.* 32.3 (2023), pp. 1–46.

[PAT24] U. Pavloff, Y. Amoussou-Guenou, and S. Tucci-Piergiovanni. "Byzantine attacks exploiting penalties in Ethereum PoS". In: *IEEE/IFIP DSN*. 2024, pp. 53–65.

[Pig+25] M. Pigaglio, O. Ascigil, M. Król, F. Lange, K. Peeroo, S. Rene, R. Sadre, V. Stankovic, and E. Rivière. "PANDAS: Peer-to-peer, Adaptive Networking Allowing Data Availability Sampling within Ethereum Consensus Timebounds". In: *Middleware*. 2025, pp. 167–179.

[PKR23] M. Pigaglio, M. Król, and E. Rivière. "Exploring locality in Ethereum transactions". In: *IEEE BRAINS*. 2023, pp. 1–8.

[Qi+26] J. Qi, D. Ding, J. Li, J. Cao, Y.-C. Chen, G. Xue, and S. Liu. "BIND: Enabling Continuous Transaction Processing During Account Migration in Sharded Blockchains". In: *ACM Web Conf.* 2026, pp. 5547–5556.

[Rou24] T. Roughgarden. "Transaction Fee Mechanism Design". In: *J. ACM* (2024), pp. 1–25.

[Sch+22] C. Schwarz-Schilling, J. Neu, B. Monnot, A. Asgaonkar, E. N. Tas, and D. Tse. "Three Attacks on Proof-of-Stake Ethereum". In: *Financial Cryptography and Data Security (FC)*. 2022, pp. 560–576.

[Sch90] F. B. Schneider. "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial". In: *ACM Computing Surveys* 22.4 (1990), pp. 299–319.

[SZ15] Y. Sompolinsky and A. Zohar. "Secure High-Rate Transaction Processing in Bitcoin". In: *Financial Cryptography and Data Security (FC)*. 2015, pp. 507–527.

[Vyz+20] D. Vyzovitis, Y. Napora, D. McCormick, D. Dias, and Y. Psaras. *GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks*. arXiv:2007.02754. 2020.

[Wan+19] G. Wang, Z. J. Shi, M. Nixon, and S. Han. "SoK: Sharding on Blockchain". In: *ACM AFT*. 2019, pp. 41–61.

[Woo14] G. Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Ethereum Yellow Paper, `https://ethereum.github.io/yellowpaper/paper.pdf`. 2014.

[WW19] J. Wang and H. Wang. "Monoxide: Scale out blockchains with asynchronous consensus zones". In: *USENIX NSDI*. 2019, pp. 95–112.

[Yin+19] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. "HotStuff: BFT Consensus with Linearity and Responsiveness". In: *ACM PODC*. 2019, pp. 347–356.

[ZMR18] M. Zamani, M. Movahedi, and M. Raykova. "RapidChain: Scaling Blockchain via Full Sharding". In: *ACM CCS*. 2018, pp. 931–948.

[ZPY23] Y. Zhang, S. Pan, and J. Yu. "TxAllo: Dynamic Transaction Allocation in Sharded Blockchain Systems". In: *IEEE ICDE*. 2023, pp. 721–733.